Università di Genova

UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MODELLING AND CONTROL OF MANIPULATORS

Third Assignment

Jacobian Matrices and Inverse Kinematics

Author:

Antonio Zerbato
Luca Allegri
Laura Andrea

Student ID:

s8999827
s7905713
s4887380

Professors:

Enrico Simetti
Giorgio Cannata

Tutors:

Luca Tarasi
Simone Borelli

December 15, 2025

Contents

Mathematical expression	Definition	MATLAB expression
$< w >$	World Coordinate Frame	w
${}^{a}_{b}R$	Rotation matrix of frame $< b >$ with respect to frame $< a >$	aRb
${}^{a}_{b}T$	Transformation matrix of frame $< b >$ with respect to frame $< a >$	aTb
${}^{a}O_{b}$	Vector defining frame $< b >$ wit respect to frame $< a >$	aOb

Table 1: Nomenclature Table

1 Assignment description

The third assignment of Modelling and Control of Manipulators focuses on Inverse Kinematics (IK) control of a robotic manipulator.

The third assignment consists of three exercises. You are asked to:

- Download the .zip file called from the Aulaweb page of this course.
- Implement the code to solve the exercises on MATLAB by filling in the predefined files.
- Write a report motivating your answers, following the predefined format on this document.
- **Do NOT put your code in the report!**

1.1 Exercise 1

Given the geometric model of an industrial manipulator in Figure 1, you have to add a tool frame. The tool frame is rigidly attached to the robot end-effector according to the following specifications: $\Gamma_{t/e} = [\pi/10, 0, \pi/6], {}^{e}O_t = [0.3, 0.1, 0]^\top (m)$ where $\Gamma_{t/e}$ represents the YPR values from end effector frame to tool frame.

To complete this task you should modify the class *geometricModel* by adding a new method called *getToolTransformWrtBase*.

1.2 Exercise 2

Implement an inverse kinematic control loop to control the tool of the manipulator. You should be able to complete this exercise by using the MATLAB classes implemented for the previous assignment (*geometricModel*,*kinematicModel*), and also you need to implement a new class *cartesianControl* (see the template attached). The initial joint position is $q = [\pi/2, -\pi/4, 0, -\pi/4, 0, 0.15, \pi/4]^\top$, expressed in radians and meters. The procedure can be split into the following phases

Q2.1 Compute the cartesian error between the robot end-effector frame ${}^{b}_{t}T$ and the goal frame ${}^{b}_{g}T$.

The goal frame must be defined knowing that:

- The goal position with respect to the base frame is ${}^{b}O_g = [0.2, -0.7, 0.3]^\top$ (m)
- The goal frame is rotated by the YPR angles $\Gamma_g = [0, 1.57, 0]^\top$ (rad) with respect to the base frame.

Q2.2 Compute the desired angular and linear reference velocities of the tool with respect to the base: ${}^{b}\nu^{*}_{t/b} = \begin{bmatrix} \kappa_a & 0 \\ 0 & \kappa_l \end{bmatrix} \cdot {}^{b}e$, such that $\kappa_a = 0.8$,$\kappa_l = 0.8$ is the gain.

Q2.3 Compute the desired joint velocities $\dot{\tilde{q}}$

Q2.4 Simulate the robot motion by implementing the function: *"KinematicSimulation()"* for integrating the joint velocities in time.

Q2.5 Compute the end effector and tool linear and angular velocities with respect to the base frame projected on the base frame.

Remark 1: All the methods must be implemented for a generic serial manipulator. For instance, joint types, and the number of joints should be parameters.

Remark 2: The class *plotManipulators* is provided for plot generation. You must not modify its implementation, or its calls in *main.m.*

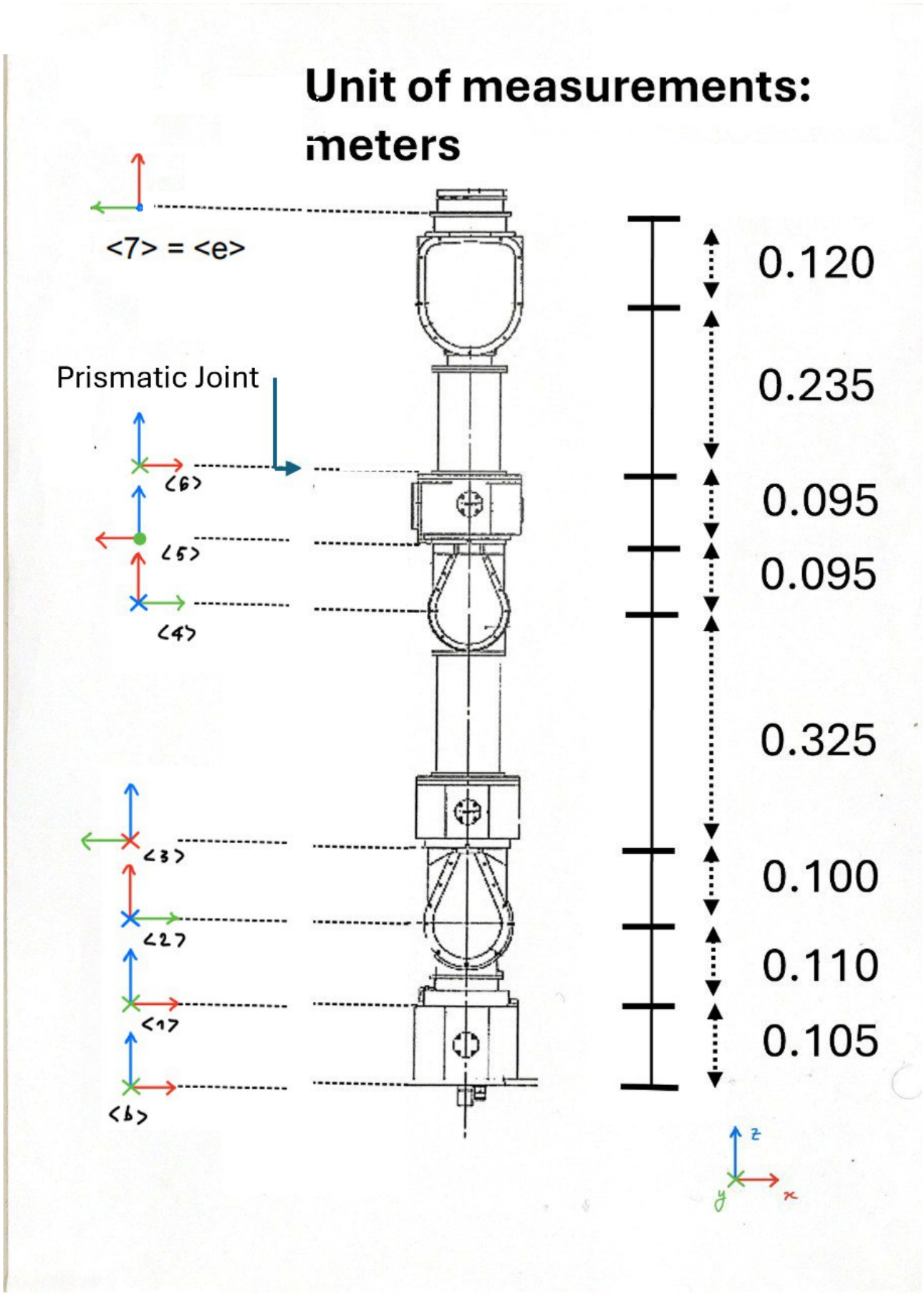

Figure 1: Manipulator CAD

2 Exercise 1

The objective of this exercise is to extend the geometric model of a serial industrial manipulator by introducing a tool reference frame rigidly attached to the robot end-effector. The tool frame is defined with respect to the end-effector frame through a fixed orientation and position, and the final goal is to compute the homogeneous transformation matrix from the robot base frame to the tool frame.

The pose of the tool frame (t) with respect to the end-effector frame (e) is given by a Yaw–Pitch–Roll orientation: $\Gamma_{t/e} = \begin{bmatrix} \frac{\pi}{10} & 0 & \frac{\pi}{6} \end{bmatrix}$ and a translation vector expressed in the end-effector frame: ${}^{e}\mathbf{O}_t = \begin{bmatrix} 0.3 \\ 0.1 \\ 0 \end{bmatrix}$ m

The YPR angles are converted into a rotation matrix ${}^{e}R_t$ using the custom function `YPRToRot` previously implemented. By combining the rotation and translation components, the homogeneous transformation from the end-effector frame to the tool frame is obtained as:

$$ {}^{e}T_t = \begin{bmatrix} {}^{e}R_t & {}^{e}\mathbf{O}_t \\ \mathbf{0}^\top & 1 \end{bmatrix} $$

In order to model a tool rigidly attached to the robot end-effector, the geometric model was extended by introducing the constant homogeneous transformation ${}^{e}T_t$.

To support this extension, the constructor of the `geometricModel` class was updated to take three input arguments instead of two: the initial link transformations, the joint type vector, and the tool transformation ${}^{e}T_t$.

This design choice is useful for implementing the `getToolTransformWrtBase` method of the `geometricModel` class. Within this function, the transformation from the base frame to the tool frame is obtained by first computing the transformation from the base to the end-effector, and then multiplying it by ${}^{e}T_t$ as follows:

$$ {}^{0}T_t = {}^{0}T_e \, {}^{e}T_t $$

3 Exercise 2

3.1 Q2.1

The first task of the second exercise focuses on computing the diplacement error between the tool and the goal frame, expressed with respect to the base frame b. The error, e, takes into account a distance part derived by the difference between the two origins and a misalignment part due to the different orientation of the two frames.

$$ {}^{b}\boldsymbol{e} = \begin{bmatrix} {}^{b}\boldsymbol{\rho} \\ {}^{b}\boldsymbol{r} \end{bmatrix} $$

In particular, since the goal frame is stationary, the linear error term ${}^{b}\boldsymbol{r}$ is computed as the difference between the origins of the goal frame and the tool frame:

$$ {}^{b}\boldsymbol{r} = {}^{b}\boldsymbol{O}_g - {}^{b}\boldsymbol{O}_t $$

where the origins are extracted as the first three elements of the last column of the corresponding homogeneous transformation matrices with respect to the base frame.

Instead, the angular error term ${}^{b}\boldsymbol{\rho}$ is defined as the product of the rotation axis h and the rotation angle θ, obtained from the rotation matrix that aligns the tool frame with the goal frame, expressed in the base frame:

$$ \boldsymbol{R_{err}} = {}^{b}\boldsymbol{R}_g - {}^{b}\boldsymbol{R}_t^T $$

The corresponding axis–angle pair (h, θ) associated with the rotation matrix R_{err} is computed using the `RotToAngleAxis` custom function, implemented in the provious assingment .

3.2 Q2.2

To compute the desired angular and linear velocity of the tool with respect to the base frame, we focus on the control law that combines the distance zeroing and the misalignment problem:

$$\dot{\mathbf{x}}_{\boldsymbol{t/b}} = \boldsymbol{\Lambda}\, {}^{b}\mathbf{e} + \dot{\mathbf{x}}_{\boldsymbol{g/b}}$$

Since the actual Cartesian velocity of the goal, $\dot{\mathbf{x}}_{g/b}$, is zero (the goal frame is stationary), the formula reduces to: $\dot{\mathbf{x}}_{t/b} = \boldsymbol{\Lambda}\, {}^{b}\mathbf{e}$, where:

- $\dot{\mathbf{x}}_{t/b}$ is the desired Cartesian velocity, a 6×1 vector composed as: $\begin{bmatrix} {}^{\boldsymbol{b}}\boldsymbol{\omega}_{\boldsymbol{t/b}} \\ {}^{\boldsymbol{b}}\mathbf{v}_{\boldsymbol{t/b}} \end{bmatrix}$
- $\boldsymbol{\Lambda}$ is the Cartesian gain matrix, defined as: $\begin{bmatrix} \boldsymbol{k_a I_3} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{k_l I_3} \end{bmatrix}$
- ${}^{b}\mathbf{e}$ is the cartesian error between the robot end-effector frame and the goal frame, expressed with respect to the base frame.

Hence, the final product to obtain the desired velocities of the tool with respect to the base is:

$$\dot{\mathbf{x}}_{t/b} = \begin{bmatrix} k_a \mathbf{I}_3 & \mathbf{0} \\ \mathbf{0} & k_l \mathbf{I}_3 \end{bmatrix} \begin{bmatrix} {}^{b}\boldsymbol{\rho} \\ {}^{b}\boldsymbol{r} \end{bmatrix}$$

By imposing the gains $k_a = 0.8$ and $k_l = 0.8$, after running the simulation the final Cartesian error and desired Cartesian velocity are:

$${}^{b}\mathbf{e} = \begin{bmatrix} 0.0084 \\ 0.0018 \\ 0.0085 \\ 0.0035 \\ 0.0057 \\ 0.0007 \end{bmatrix} \qquad \dot{\mathbf{x}}_{t/b} = \begin{bmatrix} 0.0067 \\ 0.0014 \\ 0.0068 \\ 0.0028 \\ 0.0045 \\ 0.0005 \end{bmatrix}$$

3.3 Q2.3

Before computing the desired joint velocities $\dot{\boldsymbol{q}}$, it is important to focus on how the Jacobian is updated at each iteration using the `updateJacobian` function of the `geometricMedel` class. Since the tool frame is rigidly attached to the end–effector, the Jacobian should not be computed at the end–effector frame, but the control is applied at the tool frame, which is translated with respect to the end–effector. Therefore, the Jacobian of the tool is obtained as:

$$\boldsymbol{J_t} = \mathrm{B} \cdot \boldsymbol{J_e}$$

where $\boldsymbol{B}$ is the Rigid Body Jacobian, defined as:

$$\begin{bmatrix} \boldsymbol{I_3} & \mathbf{0} \\ [{}^{b}\boldsymbol{r_{et}}\times]^{\boldsymbol{T}} & \boldsymbol{I_3} \end{bmatrix}$$

which accounts for the rigid translation or fixed offset $\boldsymbol{r_{et}}$ from the end–effector to the tool.

Once the Jacobian is computed, the differential kinematic relationship between the Cartesian velocity of the tool and the joint velocities is expressed as:

$$\dot{\mathbf{x}}_{t/b} = \mathbf{J}(\mathbf{q})\, \dot{\mathbf{q}}$$

So, the inverse kinematics problem is then solved using the Moore–Penrose pseudoinverse of the Jacobian:

$$\dot{\mathbf{q}} = \mathbf{J}^{\dagger} \dot{\mathbf{x}}_{t/b}$$

The computed joint velocities are integrated in a time loop to simulate the robot motion over time.

3.4 Q2.4

This part of the assignment focuses on the simulation of the robot motion by implementing the `KinematicSimulation` function, which integrates the joint velocities over time. The function takes as input parameters:

- q the current robot configuration
- q_dot the joints velocity
- dt sampling time
- q_min lower joints bounds
- q_max upper joints bounds

The output vector q_new, representing the updated joint configuration, was initially allocated with the same dimension as q. The joint configuration is then updated by performing a first-order Euler integration of the joint velocities, computed as the sum of the initial joint variable values with the product between the q_dot and the sampling time dt. Finally, a clamping operation with the lower and upper bounds is applied to each joint variable, ensuring that the resulting configuration remains both mathematically consistent and physically feasible.

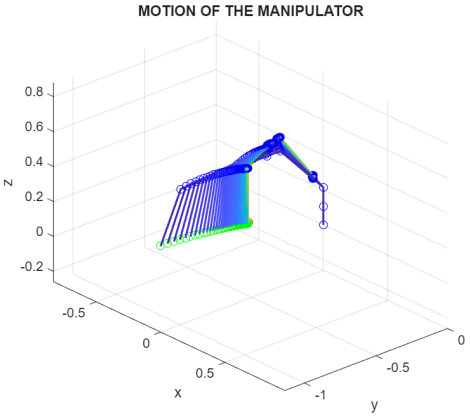

Figure 2: Motion of the manipulator

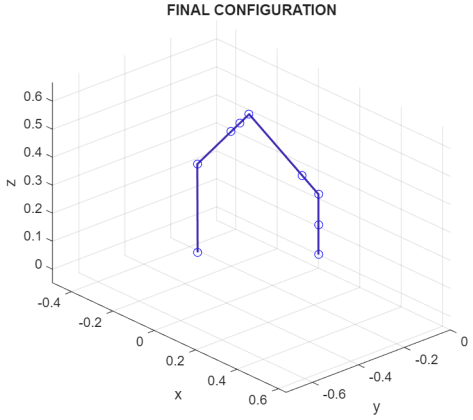

Figure 3: Final Robot Configuration

As illustrated in the above Fig: 6, the motion of the manipulator is correctly executed to reach the goal configuration, represented by a red circle. The adopted approach, described in the previous sections is purely deterministic. In particular, the manipulator tool computes a direct trajectory aimed at minimizing the task-space error and driving it to convergence in the most efficient manner. Figure 7 shows the final robot configuration obtained at the end of the simulation.

3.5 Q2.5

In this task, the objective is to compute the spatial twist (angular and linear velocity) of both the end-effector and the tool with respect to the base frame, expressed in the base frame.

Tool velocity: Given the joint velocity vector $\dot{\mathbf{q}}$, the tool twist with respect to the base is obtained through the Jacobian mapping:

$$^{b}\boldsymbol{\nu}_{t/b} = \mathbf{J}_t(\mathbf{q})\,\dot{\mathbf{q}}$$

where $\mathbf{J}_t(\mathbf{q})$ is the tool Jacobian provided by the kinematic model. The resulting twist vector is decomposed into angular and linear components:

$$^{b}\boldsymbol{\nu}_{t/b} = \begin{bmatrix} ^{b}\boldsymbol{\omega}_{t/b} \\ ^{b}\mathbf{v}_{t/b} \end{bmatrix}$$

The resulting 6×1 vector is then split into:

$$^{b}\boldsymbol{\omega}_{t/b} = \mathbf{V}_{t/b}(1{:}3), \qquad ^{b}\mathbf{v}_{t/b} = \mathbf{V}_{t/b}(4{:}6)$$

where $^{b}\boldsymbol{\omega}_{t/b}$ is the angular velocity and $^{b}\mathbf{v}_{t/b}$ is the linear velocity. The numerical values obtained are:

$$^{b}\boldsymbol{\omega}_{t/b} = 10^{-3} \begin{bmatrix} 0.0067 \\ 0.0014 \\ 0.0068 \end{bmatrix} \qquad ^{b}\mathbf{v}_{t/b} = \begin{bmatrix} 0.0028 \\ 0.0045 \\ 0.0005 \end{bmatrix}$$

End-effector velocity: Similarly, the end-effector twist is computed by first obtaining the Jacobian of the end-effector link with respect to the base and then applying the same Jacobian mapping:

$$^{b}\boldsymbol{\nu}_{e/b} = \mathbf{J}_{e}(\mathbf{q})\,\dot{\mathbf{q}}$$

The twist vector is again decomposed into angular and linear components:

$$^{b}\boldsymbol{\omega}_{e/b} = \mathbf{V}_{e/b}(1{:}3), \qquad ^{b}\mathbf{v}_{e/b} = \mathbf{V}_{e/b}(4{:}6)$$

The numerical values obtained for the end-effector velocities are:

$$^{b}\boldsymbol{\omega}_{e/b} = 10^{-3}\begin{bmatrix}0.0067\\0.0014\\0.0068\end{bmatrix} \qquad ^{b}\mathbf{v}_{e/b} = \begin{bmatrix}0.0033\\0.0024\\0.0006\end{bmatrix}$$

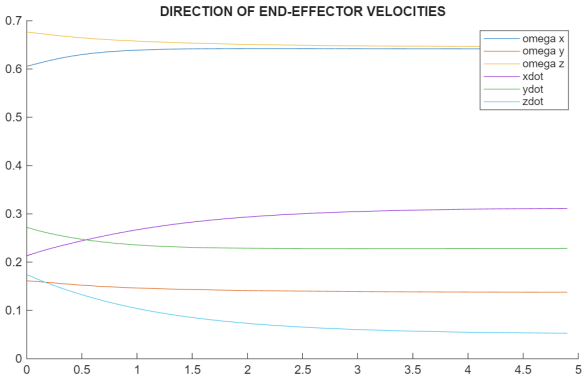

Figure 4: Direction of End-Effector

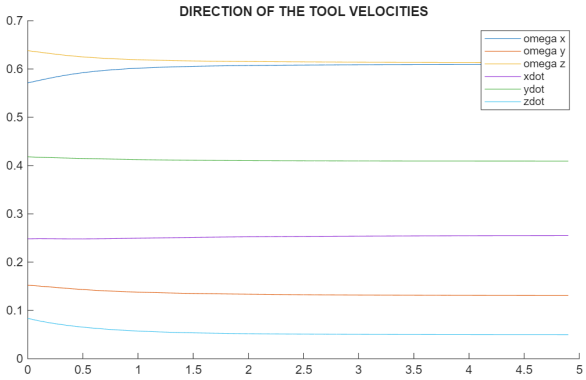

Figure 5: Direction of Tool

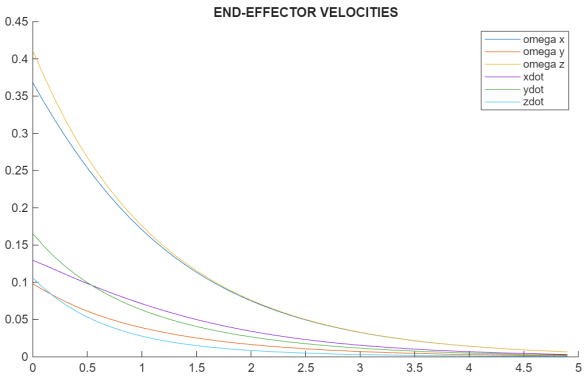

Figure 6: End-Effector velocities

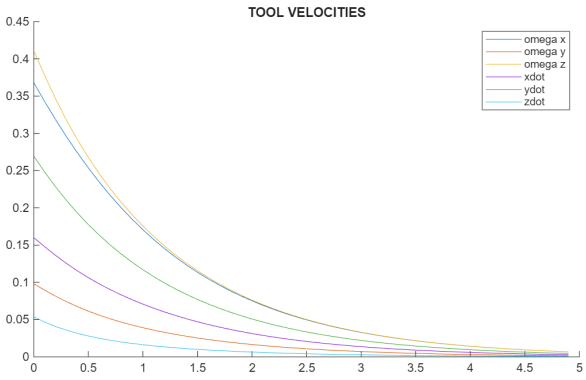

Figure 7: Tool velocities